Devixo Solutions

LIBRARY MANAGEMENT SYSTEM

C++ Programming Internship
TASK 02 — PROJECT REPORT
Submitted By: Ayesha Raheela
Internship Domain: C++ Programming
Organization: Devixo Solutions
Project Type: Console-Based Application
Programming Language: C++
Development Environment: Code Utility Online C++ Compiler
Submission Date: 26 October 2026

Project Documentation and Implementation Report

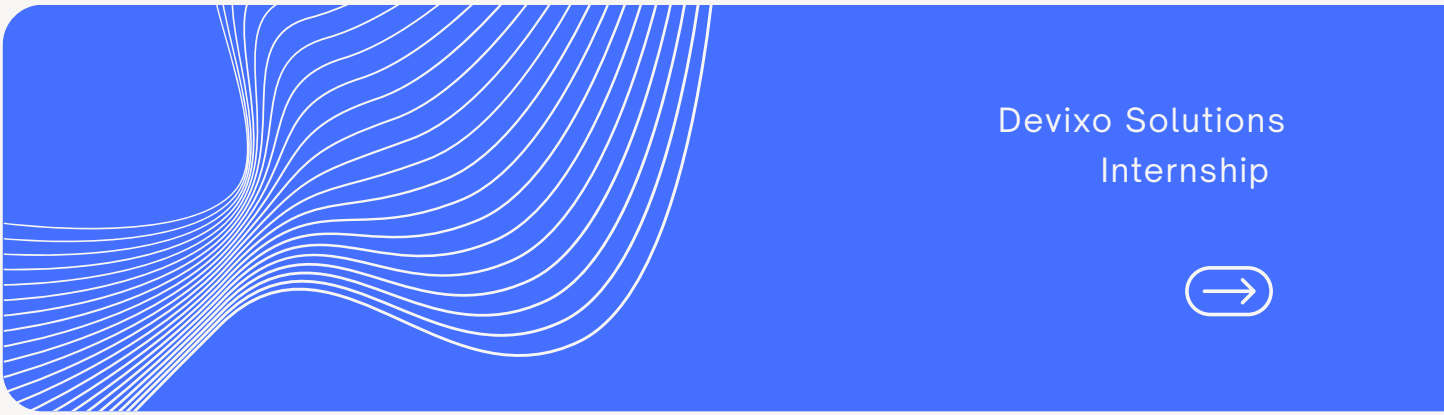


Table of Contents

1. Introduction — Page 3
2. Project Objective---Page 4
3. Problem Statement and Project Scope — Page 5-6
4. Technologies Used — Page 7
5. Implementation Methodology — Page 8
6. Program Structure and Data Representation — Page 9
7. Code Screenshots ---Page 10-13
8. Implementation and Code Explanation — Page 14-17
9. File Handling and Data Persistence — Page 18-19
10. Input Validation and Error Handling — Page 20
11. Testing and Results — Page 21
12. Challenges Faced and Solutions — Page 22-23
13. Learning Outcomes — Page 24
14. Conclusion — Page 25
15. Project Summary --- Page 26
16. GitHub Repository and Project Details — Page 27
17. Final Statement — Page 28

Introduction

3

The Library Management System is a console-based application developed in C++ as part of Devixo Solutions Internship Task 02.

The purpose of this project is to demonstrate the practical application of fundamental C++ programming concepts by developing a system that manages library book records through a menu-driven interface.
The application provides operations for adding new books, viewing existing records, searching for books by title or author, updating book information, deleting records, issuing books, and returning issued books.
It also incorporates file handling to save and retrieve library records using a text file named book_data.txt.
The project uses classes and objects, constructors, functions, vectors, loops, conditional statements, strings, input validation, and file input/output operations.
This project provides practical experience in designing a structured program, organizing functionality into manageable components, maintaining records, handling user input, and testing application behavior.



Inclusivity



Transparency



Accountability

Project Objective

The primary objective of this project is to develop a functional Library Management System using C++.

The specific objectives are:

- To create a user-friendly, menu-driven console application.
- To implement operations for adding, viewing, searching, updating, and deleting book records.
- To implement book issue and return functionality.
- To use classes and objects to organize book information and library operations.
- To use a vector to maintain a collection of book records.
- To implement file handling for saving and loading library data.
- To validate user input and handle common errors.
- To test the application's functionality and verify its behavior.



PROBLEM STATEMENT AND SCOPE

Problem Statement

Libraries require an organized way to maintain information about books, locate specific titles, update records, and track the availability of books.

Managing these operations manually can become time-consuming and may lead to inconsistent or outdated records.

The objective of this project is to address these basic requirements by developing a C++ application that allows users to manage book records through a simple console menu.

The system organizes book information into individual records and provides operations for maintaining and retrieving these records. It also includes file handling to retain data beyond the program's immediate execution, provided the data file remains accessible.



Project Scope and Features

The application includes the following features:

1. Add Book
Allows the user to enter the required information for a new book and add the record to the library collection.
2. View Books
Displays the book records maintained by the application, helping users review the library collection.
3. Search Book
Searches for books by title or author and displays matching records when found.
4. Update Book Information
Allows the user to modify the information associated with an existing book record.
5. Delete Book
Removes a selected book record from the library collection when the requested record exists.
6. Issue Book
Processes a book issue request and updates the book's availability or issue status according to the implemented logic.
7. Return Book
Processes a return request and updates the book's status according to the program's current record.
8. Save and Load Records
Uses book_data.txt to store and retrieve library data through file-handling operations.
9. Menu-Driven Interface
Provides an organized menu through which users can select and perform available operations.

TECHNOLOGIES USED

The project was developed using the following technologies and C++ programming concepts.

1

Programming Language

C++
C++ was used to implement the application logic, manage book records, process user input, and perform file operations.

2

Development Environment

CodeUtility Online C++ Compiler
The program was developed and tested in an online C++ environment that provided a source-code editor, execution console, and workspace for project files.

3

Header Files and Libraries

The program uses the following standard C++ headers:

- iostream — for console input and output.
- string — for storing and processing textual book information.
- vector — for maintaining a dynamically sized collection of book records.
- fstream — for reading and writing data files.
- limits — for handling input-stream errors where required.
- algorithm and character-handling utilities — where used for searching and case conversion.

4

Programming Concepts

The implementation demonstrates:

- Classes and objects.
- Constructors.
- Functions and modular programming.
- Vectors and data management.
- Loops and conditional statements.
- String handling.
- Input validation.
- File handling and data persistence.

These concepts help organize the program and make its functionality easier to understand and maintain.

IMPLEMENTATION METHODOLOGY

The application was developed in stages to ensure that each major operation could be implemented and tested independently.

Step 1: Define the Book Class
A class named Book represents an individual book record. It stores the information required to identify and manage a book, including the fields defined in the source code.
Step 2: Create the Library Class
A class named Library manages the collection of books and provides the operations required for maintaining library records.
Step 3: Implement the Collection
A vector of book objects is used to maintain the collection in memory. This allows the collection to grow or shrink as records are added or deleted.
Step 4: Develop Book Management Functions
Separate functions implement adding, viewing, searching, updating, and deleting book records.
Step 5: Implement Issue and Return Operations
Functions are used to process book issue and return requests according to the availability information maintained by the program.
Step 6: Implement File Handling
The application uses file operations to save library records to book_data.txt and retrieve records when loading is supported and the file is accessible.
Step 7: Develop the Menu System
The main execution function displays the available operations and processes the user's selection until the exit option is selected.
Step 8: Test the Application
The application was tested by selecting the available menu operations and checking the resulting behavior.

The staged approach helped organize development and made it easier to examine the application's functionality.

PROGRAM STRUCTURE AND DATA REPRESENTATION

1. **Book Class**

The Book class represents a single book record. It contains the fields used by the program to identify and manage each book.
A constructor initializes the book information when an object is created.

2. **Library Class**

The Library class manages the collection of books and provides functions for performing the available operations.
Its responsibilities include searching for records, displaying book information, modifying records, processing issue and return requests, and handling file input and output.

3. **Vector-Based Storage**

The application uses a vector to store book objects in memory. A vector is suitable for a collection whose size may change during execution.
When a book is added, its record can be inserted into the collection. When a record is deleted, the collection can be reduced accordingly.

4. **Function-Based Organization**

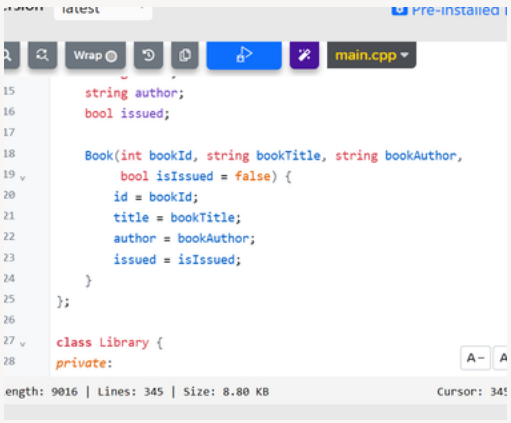
The program divides its responsibilities into functions. Examples include operations for adding, viewing, searching, updating, and deleting books, along with functions for saving and loading records.
This modular structure improves readability and reduces the need to place all application logic inside main().

5. **Main Function**

The main() function creates the library object and starts the application by calling the appropriate execution function.

Libraries, and Classes

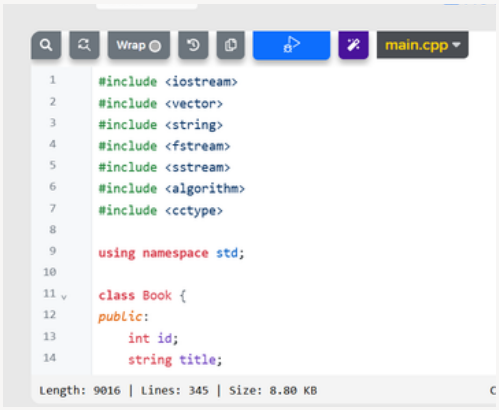
Code Screenshots:



```
    string author;
    bool issued;

    Book(int bookId, string bookTitle, string bookAuthor,
         bool isIssued = false) {
        id = bookId;
        title = bookTitle;
        author = bookAuthor;
        issued = isIssued;
    }
};

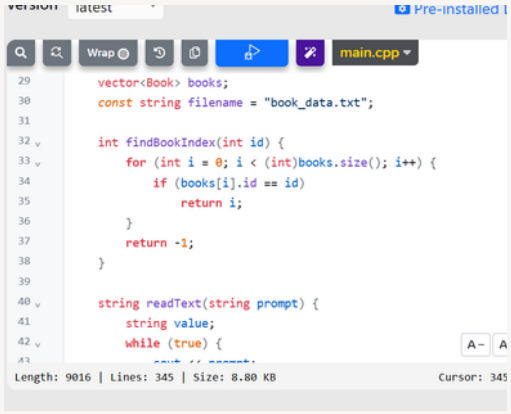
class Library {
private:
```



```
#include <iostream>
#include <vector>
#include <string>
#include <fstream>
#include <sstream>
#include <algorithm>
#include <cctype>

using namespace std;

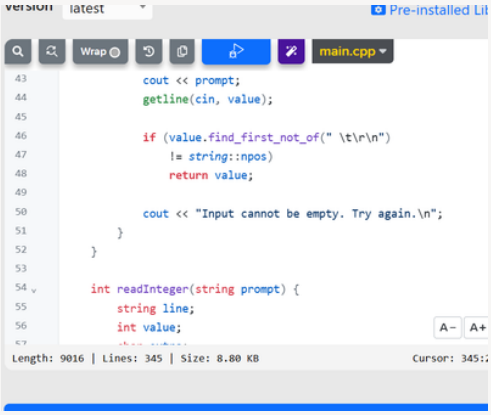
class Book {
public:
    int id;
    string title;
```



```
    vector<Book> books;
    const string filename = "book_data.txt";

    int findBookIndex(int id) {
        for (int i = 0; i < (int)books.size(); i++) {
            if (books[i].id == id)
                return i;
        }
        return -1;
    }

    string readText(string prompt) {
        string value;
        while (true) {
```

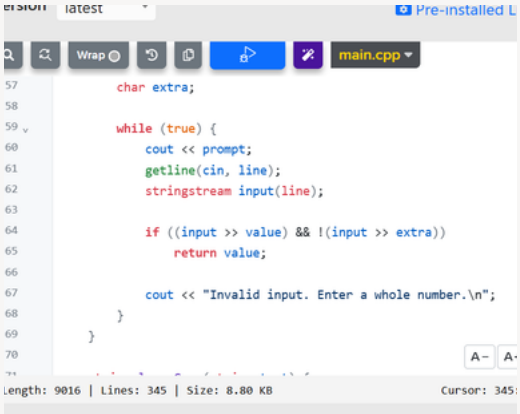


```
            cout << prompt;
            getline(cin, value);

            if (value.find_first_not_of(" \t\r\n")
                != string::npos)
                return value;

            cout << "Input cannot be empty. Try again.\n";
        }
    }

    int readInteger(string prompt) {
        string line;
        int value;
```

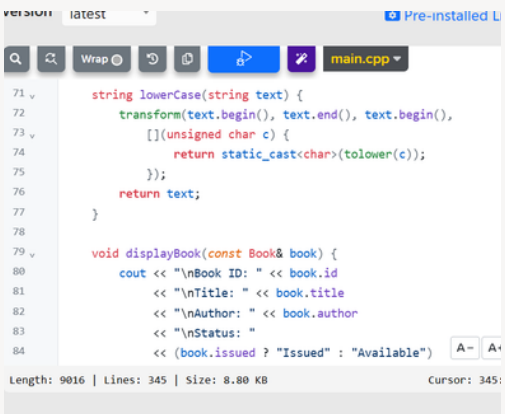


```
        char extra;

        while (true) {
            cout << prompt;
            getline(cin, line);
            stringstream input(line);

            if ((input >> value) && !(input >> extra))
                return value;

            cout << "Invalid input. Enter a whole number.\n";
        }
    }
```



```
    string lowerCase(string text) {
        transform(text.begin(), text.end(), text.begin(),
            [](unsigned char c) {
                return static_cast<char>(tolower(c));
            });
        return text;
    }

    void displayBook(const Book& book) {
        cout << "\nBook ID: " << book.id
             << "\nTitle: " << book.title
             << "\nAuthor: " << book.author
             << "\nStatus: "
             << (book.issued ? "Issued" : "Available")
```

Input Validation Functions

Code Screenshots:

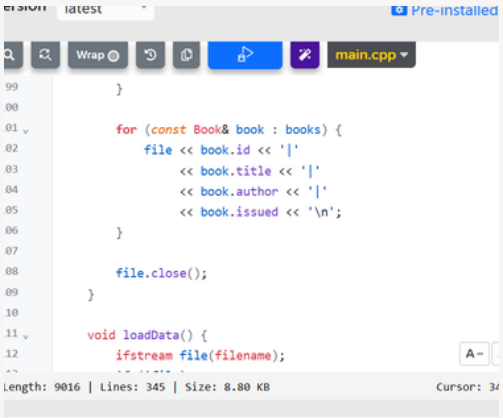


```
<< "\n----------\n";
}

public:
    Library() {
        loadData();
    }

    void saveData() {
        ofstream file(filename);

        if (!file) {
            cout << "Error: Could not save book data.\n";
            return;
```



```
    }

    for (const Book& book : books) {
        file << book.id << '|'
             << book.title << '|'
             << book.author << '|'
             << book.issued << '\n';
    }

    file.close();
}

void loadData() {
    ifstream file(filename);
```



```
            !getline(record, issuedText))
            continue;

        stringstream idInput(idText);
        stringstream issuedInput(issuedText);

        if (!(idInput >> id) ||
            !(issuedInput >> issued) ||
            (issued != 0 && issued != 1) ||
            id <= 0 ||
            title.find('|') != string::npos ||
            author.find('|') != string::npos)
            continue;
```

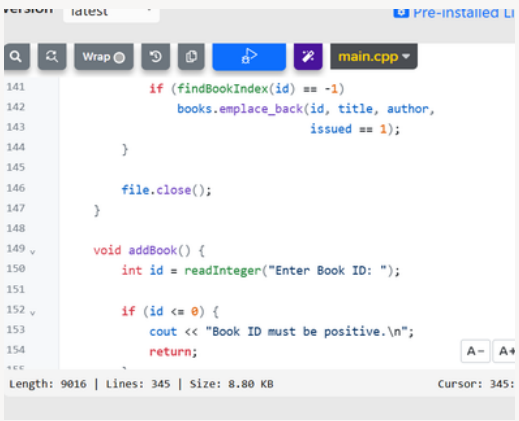


```
    if (!file)
        return;

    string line;

    while (getline(file, line)) {
        stringstream record(line);
        string idText, title, author, issuedText;
        int id;
        int issued;

        if (!getline(record, idText, '|') ||
            !getline(record, title, '|') ||
            !getline(record, author, '|') ||
```

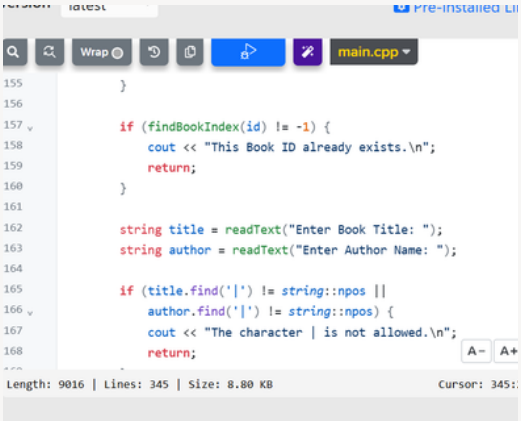


```
            if (findBookIndex(id) == -1)
                books.emplace_back(id, title, author,
                                   issued == 1);
        }

        file.close();
    }

    void addBook() {
        int id = readInteger("Enter Book ID: ");

        if (id <= 0) {
            cout << "Book ID must be positive.\n";
            return;
```



```
    }

    if (findBookIndex(id) != -1) {
        cout << "This Book ID already exists.\n";
        return;
    }

    string title = readText("Enter Book Title: ");
    string author = readText("Enter Author Name: ");

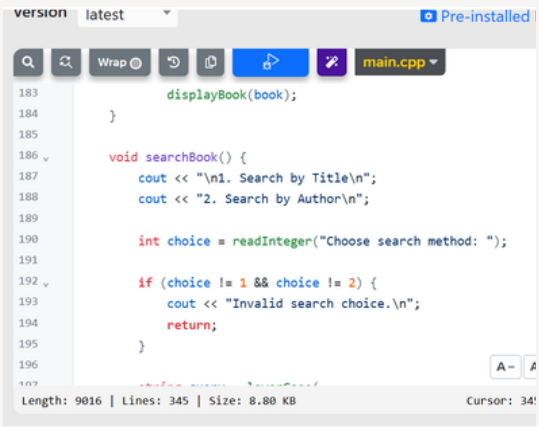
    if (title.find('|') != string::npos ||
        author.find('|') != string::npos) {
        cout << "The character | is not allowed.\n";
        return;
```

Input Validation Functions

Code Screenshots:



```
169         }
170
171         books.emplace_back(id, title, author);
172         saveData();
173         cout << "Book added successfully!\n";
174     }
175
176     void viewBooks() {
177         if (books.empty()) {
178             cout << "No books are currently recorded.\n";
179             return;
180         }
181
182         for (const Book& book : books)
```



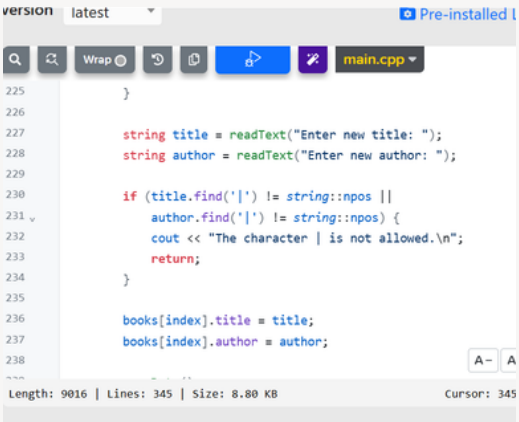
```
183             displayBook(book);
184     }
185
186     void searchBook() {
187         cout << "\n1. Search by Title\n";
188         cout << "2. Search by Author\n";
189
190         int choice = readInteger("Choose search method: ");
191
192         if (choice != 1 && choice != 2) {
193             cout << "Invalid search choice.\n";
194             return;
195         }
196
```



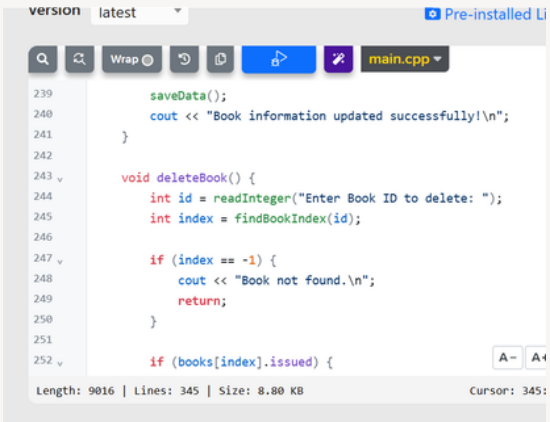
```
197         string query = lowerCase(
198             readText(choice == 1
199                 ? "Enter title or title keyword: "
200                 : "Enter author name or keyword: "));
201
202         bool found = false;
203
204         for (const Book& book : books) {
205             string field = lowerCase(
206                 choice == 1 ? book.title : book.author);
207
208             if (field.find(query) != string::npos) {
209                 displayBook(book);
210                 found = true;
```



```
211             }
212         }
213
214         if (!found)
215             cout << "No matching book found.\n";
216     }
217
218     void updateBook() {
219         int id = readInteger("Enter Book ID to update: ");
220         int index = findBookIndex(id);
221
222         if (index == -1) {
223             cout << "Book not found.\n";
224             return;
```



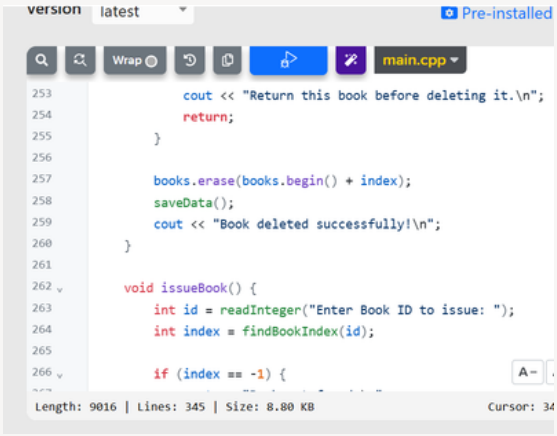
```
225         }
226
227         string title = readText("Enter new title: ");
228         string author = readText("Enter new author: ");
229
230         if (title.find('|') != string::npos ||
231             author.find('|') != string::npos) {
232             cout << "The character | is not allowed.\n";
233             return;
234         }
235
236         books[index].title = title;
237         books[index].author = author;
238
```

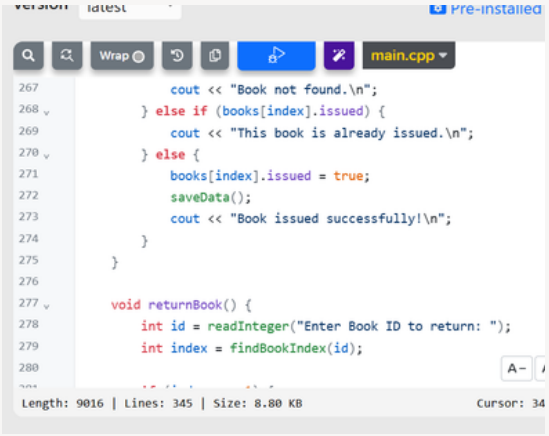


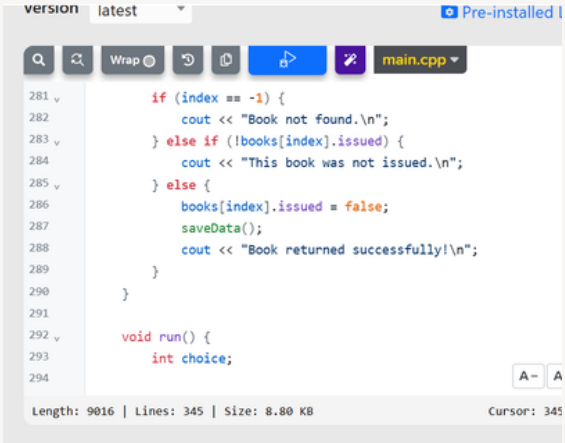
```
239         saveData();
240         cout << "Book information updated successfully!\n";
241     }
242
243     void deleteBook() {
244         int id = readInteger("Enter Book ID to delete: ");
245         int index = findBookIndex(id);
246
247         if (index == -1) {
248             cout << "Book not found.\n";
249             return;
250         }
251
252         if (books[index].issued) {
```

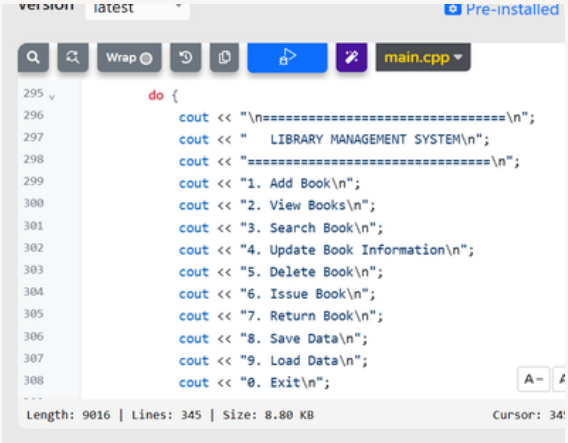
Menu Loop

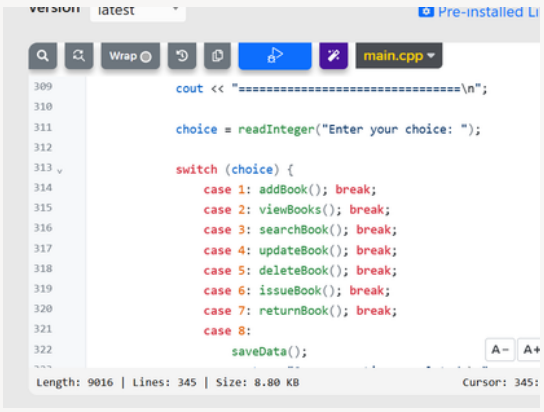
Code Screenshots:











IMPLEMENTATION AND CODE EXPLANATION

The program is organized into separate functions to improve readability, testing, and maintainability.

1. **Add Book Function**

The addBook() function collects the information required to create a new book record. The program processes the entered information and adds the record to the collection when the operation is valid.

```
================================
   LIBRARY MANAGEMENT SYSTEM
================================
1. Add Book
2. View Books
3. Search Book
4. Update Book Information
5. Delete Book
6. Issue Book
7. Return Book
8. Save Data
9. Load Data
0. Exit
================================
```

```
Enter your choice: 1
Enter Book ID: 102
Enter Book Title: C++ Programming OPP
Enter Author Name: Zeeshan Bashir
Book added successfully!
```

IMPLEMENTATION AND CODE EXPLANATION

2. View Books Function

The viewBooks() function displays the book records currently maintained by the application. If the collection is empty, the program should display an appropriate message.

```
================================
   LIBRARY MANAGEMENT SYSTEM
================================
1. Add Book
2. View Books
3. Search Book
4. Update Book Information
5. Delete Book
6. Issue Book
7. Return Book
8. Save Data
9. Load Data
0. Exit
================================
Enter your choice: 2
```

```
================================
Enter your choice: 2

Book ID: 101
Title: C++ Programming
Author: Zeeshan
Status: Available
----------------------------

Book ID: 102
Title: C++ Programming OPP
Author: Zeeshan Bashir
Status: Available
----------------------------
```

3. Search Book Function

The searchBook() function searches for books using title or author information. Where case-insensitive matching is implemented, uppercase and lowercase differences do not prevent a match.

```
================================
   LIBRARY MANAGEMENT SYSTEM
================================
1. Add Book
2. View Books
3. Search Book
4. Update Book Information
5. Delete Book
6. Issue Book
7. Return Book
8. Save Data
9. Load Data
0. Exit
================================
Enter your choice: 3
```

```
Enter your choice: 3

1. Search by Title
2. Search by Author
Choose search method: 2
Enter author name or keyword: Zeeshan Bashir

Book ID: 102
Title: C++ Programming OPP
Author: Zeeshan Bashir
Status: Available
----------------------------
```

IMPLEMENTATION AND CODE EXPLANATION

4. Update Book Function

The updateBook() function locates the selected record and modifies the relevant book information according to the user's input.

```
==============================
   LIBRARY MANAGEMENT SYSTEM
==============================
1. Add Book
2. View Books
3. Search Book
4. Update Book Information
5. Delete Book
6. Issue Book
7. Return Book
8. Save Data
9. Load Data
0. Exit
==============================
Enter your choice: 4
```

```
==================================
Enter your choice: 4
Enter Book ID to update: 101
Enter new title: Maths
Enter new author: Zeeshan Bashir
Book information updated successfully!
```

5. Delete Book Function

The deleteBook() function removes the selected record from the collection when the requested book exists.

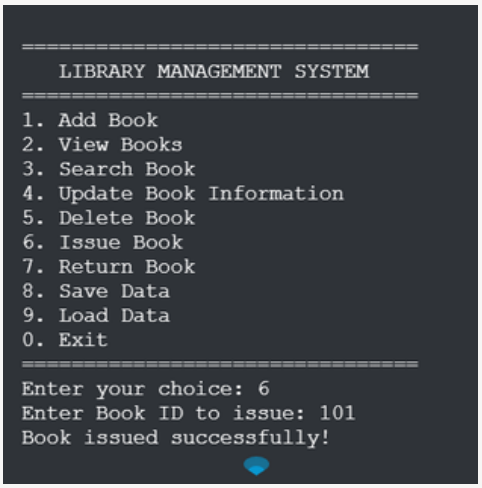
```
==============================
   LIBRARY MANAGEMENT SYSTEM
==============================
1. Add Book
2. View Books
3. Search Book
4. Update Book Information
5. Delete Book
6. Issue Book
7. Return Book
8. Save Data
9. Load Data
0. Exit
==============================
Enter your choice: 5
```

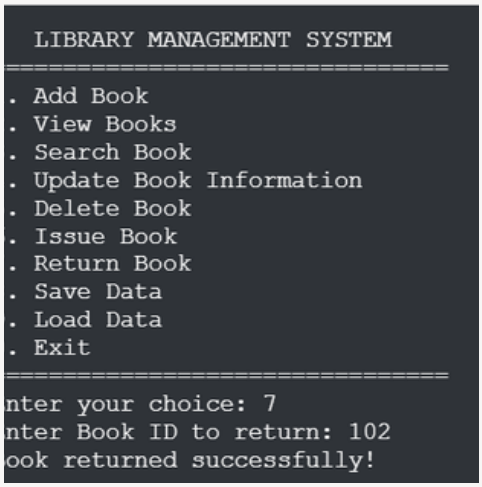
Explanation:
These functions demonstrate how the application maintains and modifies existing book information.

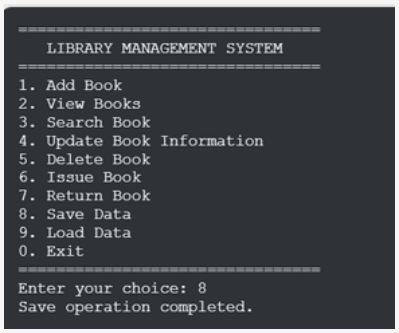
IMPLEMENTATION AND CODE EXPLANATION

6. Issue and Return Functions

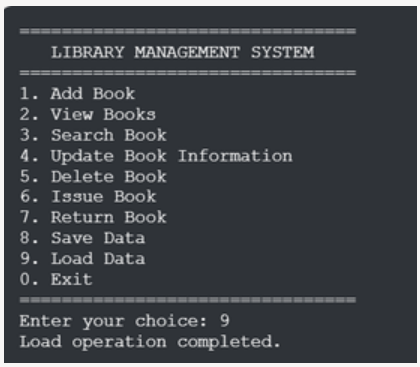
The issueBook() function processes requests to issue books. The returnBook() function processes requests to return them. The operations use the book's current status to determine how the request should be handled, according to the implemented logic.

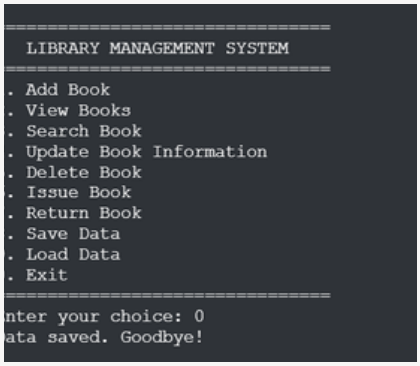






Save Data, Load Data & Exit





FILE HANDLING AND DATA PERSISTENCE

File handling is an important feature of the Library Management System because it allows records to be stored outside the program's temporary memory.
The application uses a text file named book_data.txt for record storage.

1. **Saving Records**

The saveData() function writes library records to the data file using C++ file output facilities.
The saved information follows the format expected by the program, allowing book records to be reconstructed during a later loading operation.

2. **Loading Records**

The loadData() function reads previously stored information from the data file when loading is performed and the file is available.
The program processes the stored information and restores the relevant book records in memory.

3. **Verifying File Persistence**

During testing, the program's file-handling functionality was checked, and the book_data.txt file was saved.
The report should describe the exact persistence test you performed. If you confirmed that saved records were successfully loaded during a later execution, include that result. Otherwise, report only that the file was created and could be opened.

4. **Importance of File Handling**

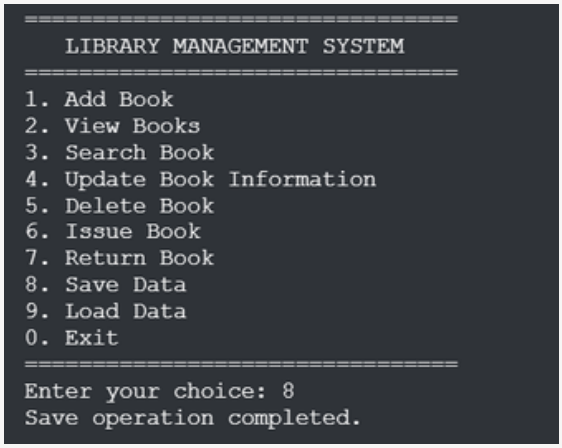
Without file storage, records held only in memory may be lost when the program terminates.
Saving records to a text file helps retain information between executions, provided the file is written successfully and remains accessible.

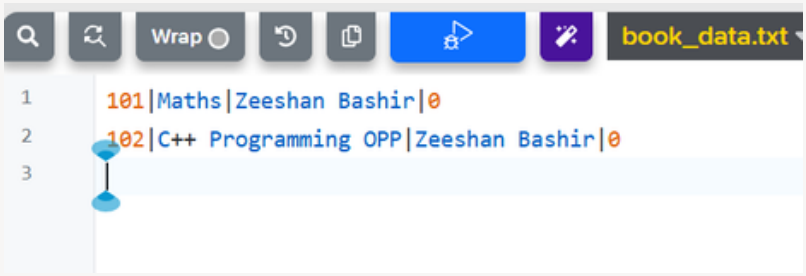
FILE HANDLING AND DATA PERSISTENCE

5. **Online Compiler Considerations**

Online compilers may use temporary or persistent workspaces. Consequently, successful file operations during one run do not guarantee that records will remain available indefinitely.

The source code and important project files should also be maintained in the GitHub repository.





Explanation: These screenshots provide evidence of the implemented file operations and the observed data-storage behavior.

INPUT VALIDATION AND ERROR HANDLING

Input validation helps ensure that the program receives acceptable data and responds appropriately to common errors.

1. **Invalid Numeric Input**

The application handles invalid numeric input according to its input-validation logic. This helps prevent inappropriate values from being processed as valid menu selections.

2. **Menu Validation**

The menu-driven interface processes the user's selection and directs execution to the corresponding operation. Invalid choices should be handled with an appropriate message.

3. **Missing Book Records**

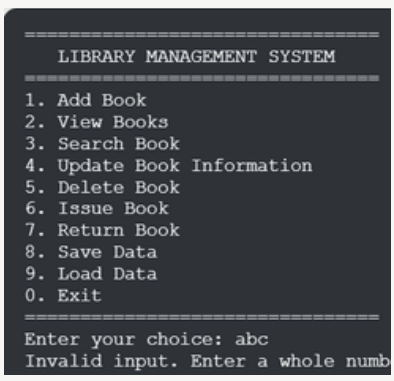
When a user searches for or attempts to update or delete a book that does not exist, the program should display a suitable message rather than modifying an unrelated record.

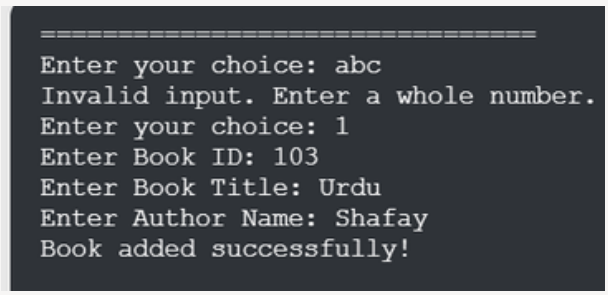
4. **Book Availability**

The issue and return operations use the current book status according to the implemented logic. This helps the application handle requests involving book availability.

5. **File-Handling Errors**

File operations depend on the availability and accessibility of book_data.txt. Appropriate checks can help identify situations in which the file cannot be opened or read.





Explanation:
Clear error messages help users understand why an operation could not be completed. Input validation and error handling also make the program easier to test and more reliable under different input conditions.

TESTING AND RESULTS

The Library Management System was tested through its console interface. The purpose of testing was to check the behavior of the implemented operations and verify the results of common user actions.

Test case	Test Scenerio	Expected Result
TC-01	Start the application	Main Menu displayed
TC-02	Add a book	Successfully added
TC-03	View books	Successfully displayed
TC-04	Search by title and author	Matching books are displayed
TC-05	Update a book	The selected record is updated
TC-06	Delete a book	The selected record is removed
TC-07	Issue a book	The issue request is processed
TC-08	Return a book	The return request is processed
TC-09	Save Data	Records are written to data file
TC-10	Load Data	Successfully retrieved
TC-11	Enter Invalid Input	Input is handled according to validation logic

CHALLENGES FACED AND SOLUTIONS

Developing and testing a menu-driven application involves several practical considerations.

1. **Managing Multiple Book Records**

Challenge: The application must maintain a collection of records while supporting additions, updates, searches, and deletions.
Solution: A vector of book objects was used to maintain the collection and support changes to its size.

2. **Organizing Program Functionality**

Challenge: Implementing all library operations in one large section of code can make the program difficult to understand and maintain.
Solution: The application was organized using the Book and Library classes and separate functions for individual operations.

3. **Searching for Book Records**

Challenge: Users need to locate records without manually reviewing the complete collection.
Solution: Search functionality was implemented to find books using title or author information.

4. **Managing Book Issue and Return Operations**

Challenge: The system needs to process issue and return requests based on the current book status.
Solution: Separate issue and return operations were implemented to manage these requests according to the program's logic.

CHALLENGES FACED AND SOLUTIONS

5. **File Persistence in an Online Environment**

Challenge: An online compiler may use a temporary or persistent workspace, affecting whether files remain available across executions.

Solution: The book_data.txt file was created and checked. Any claim of persistence across executions was based on the behavior actually verified in the workspace.

6. **Testing Menu-Driven Operations**

Challenge: Each menu operation requires the correct sequence of inputs, and incomplete input may prevent a test from finishing.

Solution: The operations were tested individually using the appropriate input sequence and checking the displayed results.

Working through these considerations improved my understanding of program structure, record management, file handling, and systematic testing.

LEARNING OUTCOMES

This project provided practical experience in applying fundamental C++ programming concepts to a functional library application.
The main learning outcomes include:

1. **Understanding Classes and Objects**

I gained practical experience in using classes to organize book information and library operations into logical components.

2. **Working with Constructors**

The project helped me understand how constructors initialize objects and establish their initial data.

3. **Developing Modular Programs**

Using separate functions for different operations improved my understanding of code organization, readability, and maintainability.

4. **Using Vectors**

The project demonstrated how vectors can store collections of objects and adjust their size when records are added or removed.

5. **Implementing File Handling**

I gained practical experience with saving and retrieving records through C++ file streams.

6. **Handling User Input**

The project highlighted the importance of validating input and responding appropriately to invalid choices and unsuccessful operations.

7. **Testing and Debugging**

Testing individual menu operations helped me examine program behavior and verify expected results.

8. **Technical Documentation**

Preparing the source code, README, screenshots, and project report provided experience in organizing and documenting a software project for review.

Overall, the project helped connect theoretical C++ concepts with practical software development.

CONCLUSION

The Library Management System was developed as part of Devixo Solutions Internship Task 02 using C++.

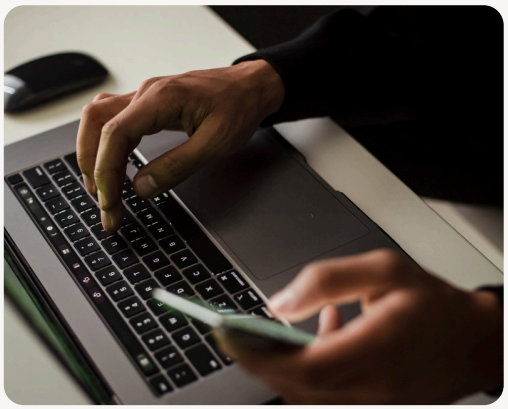
The application provides a menu-driven approach to managing book records and includes functionality for adding, viewing, searching, updating, and deleting books, as well as processing book issue and return requests.

The project demonstrates the practical application of classes, objects, constructors, functions, vectors, loops, conditional statements, strings, input validation, and file handling.

Testing the application provided an opportunity to examine its main operations, review the resulting output, and check the behavior of the file-handling functionality in the development environment.

The project strengthened my understanding of object-oriented programming, data management, program organization, testing, and technical documentation.

This work represents a useful step in my learning journey as a computer science student and provides a foundation for developing more advanced software applications in the future.



Project Summary

Items	Details
Organization	Devixo Solutions
Internship Domain	C++ Programming
Task Number	Task 02
Project Title	Library management system
Programming Language	C++
Development Environment	CodeUtility online C++ compiler
Main Source File	main.cpp
Data File	book_data.txt
Documentation	Project report in PDF format

GITHUB REPOSITORY AND PROJECT DETAILS

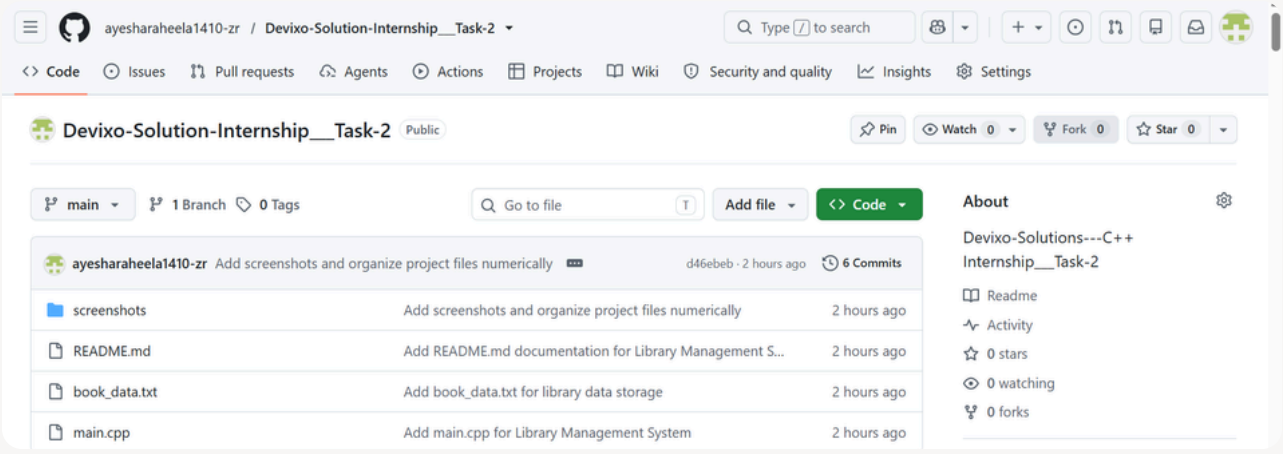
The project source code and documentation are maintained in a GitHub repository.

Repository Name: Devixo-Solutions-Internship__Task-2
Repository Link:

https://github.com/ayesharaheela1410-zr/Devixo-Solution-Internship__Task-2

The repository contains the project source code and supporting documentation. Depending on the files uploaded, it may include:

- main.cpp — C++ source code.
- book_data.txt — library data file.
- README.md — project overview and usage instructions.
- ss/ — numbered screenshots.
- Devixo_Library_Management_System_Report.pdf — final project report.



Final Statement



I am grateful for the opportunity to apply my programming knowledge to a practical project as part of the Devixo Solutions C++ Internship.
This task helped me strengthen my understanding of object-oriented programming, record management, file handling, testing, and software documentation.
The experience has encouraged me to continue developing my technical skills and work toward building more structured and reliable software applications.

Submitted by: Ayesha Raheela

End of Report

Thank You!